

Jointly Predicting Courses and Grades Using a Transformer-Based Model

Paul Savala

Mathematics, St. Edward's University, 3001 S Congress Ave, Austin,
78704, TX, USA.

Corresponding author(s). E-mail(s): psavala@stedwards.edu;

Abstract

Existing predictive models in learning analytics often treat student academic history as a simple sequence, overlooking the concurrent nature of courses taken within a semester. This simplification can lead to inaccurate performance predictions, particularly for students with heavy or challenging course loads. This paper introduces a **TR**ansformer for **A**cademic **C**ourse-grade **E**stimation (TRACE) that addresses this limitation by jointly predicting both the set of courses a student will take and their corresponding grades for an upcoming semester. Our approach encodes courses on a per-semester basis to capture the effects of course concurrency and utilizes a novel loss function combining course-set prediction with grade prediction. We demonstrate that predicting courses taken in addition to the grades in those courses leads to significant improvements in prediction quality. Trained on ten years of institutional data, our joint prediction model reduces mean absolute error by nearly 50% compared to an identical architecture that predicts grades alone. The model also outperforms traditional LSTM-based sequential models, as well as graph neural network-based approaches, and offers natural ways to incorporate student attribute data. This work demonstrates the utility of modern neural architectures for creating interpretable models that can be adapted to new institutions via retraining and recalibration, as well as the importance of key techniques, such as predicting courses taken during training. We discuss how this model could be incorporated into early detection systems at institutions of higher education.

Keywords: Learning analytics, educational data mining, predictive models, Transformer model

1 Introduction

1.1 Predictive models in LA

The field of learning analytics (LA) has long been interested in the ability to predict course outcomes [1]. In particular, past work has shown that, while grades in prior semesters are strong predictors of success in future semesters [2] [3], there are also a number of other mediating factors, such as academic ability, motivation, classroom environment, home environment, television viewing, gender, and race [4].

Clearly, there is a complex interrelationship between a student’s chosen major, their course selection, and their grades in those courses in relationship to their future performance. Two students with different majors and with distinct course histories may perform very differently in the same class. In addition, temporal factors come into play. In particular, past work has shown that the temporal proximity between a prerequisite course and the follow-up has a major effect on the student’s performance [5] [6]. However, as students progress throughout their career, the knowledge and expertise needed to perform well in one course may depend on several others. For example, a computer science student taking an algorithms and data structures course may need to draw on their knowledge of discrete mathematics, as well as object-oriented programming.

In addition, course relationships are often highly complex [7]. For example, a student who performed well in a writing course may find writing chemistry lab reports simple, but may struggle with the algebra skills needed to calculate the chemical interactions. Indeed, research on students in general education biology course has shown the effects of writing skills in a laboratory setting [7]. While simple heuristics can be devised to model certain relationships (e.g. indicating when one course is a prerequisite for another), these may fail to capture the more complex relationships that affect student performance. Moreover, most students take multiple courses in any given semester. This course load is itself a variable and may affect student performance. In addition, the precise courses taken at the same time can be an influencing factor, such as for students taking multiple difficult courses simultaneously.

Besides related courses, students also need to consider the effect of course load. Students select courses according to a number of factors, such as avoiding courses that are perceived to be too difficult, learning value, and lecturer style, among other questions [8]. In addition, the concept of “Academic Momentum” suggests that higher course loads earlier in their college career can strongly influence later outcomes [9]. Because of these factors, when modeling student performance and course selection, it’s important to consider per-semester course load as factor, rather than treating each course as an independent event.

While prior studies such as [10] and [11] have incorporated concurrently taken courses through pairwise interactions or attentional modules, our approach differs by leveraging positional encodings where all courses taken in the same semester receive the same temporal vector. This design enforces a permutation-invariant representation within each semester, which we argue more accurately reflects the unordered nature of concurrent enrollments.

1.2 Research questions

This study is guided by the following research questions:

- **RQ1:** To what extent does a Transformer model that encodes semester-level concurrency improve grade prediction accuracy compared to models that treat course history as a flat sequence?
- **RQ2:** Does the joint prediction of courses and grades lead to a significant improvement in grade prediction accuracy compared to an architecture that predicts grades alone?

While Transformer architectures are well-established, their application to academic prediction requires resolving several non-trivial design choices that have not been addressed in prior work. Specifically:

- How should concurrent courses be represented in sequence models? Should the representation be as ordered sequences or unordered sets?
- Should course identity and grade be predicted jointly or separately, and how does this choice affect learned representations?
- What loss function appropriately handles set-valued predictions where order is arbitrary, while not adversely affecting training or inference time? In addition, in the learning analytics context, while order is arbitrary, the pairing of courses and grades is important, so order is only arbitrary up to (course, grade) pair. How should this be handled?

These implementation decisions substantially affect model performance, as we demonstrate empirically, yet have received little attention in the learning analytics literature.

Prior work in course and grade prediction has typically treated these as separate tasks. Recent models such as CourseBEACON [12] and PLAN-BERT [13] use sequential or masked language models for course planning or recommendation, but do not jointly predict course sets and associated performance. Our approach addresses this by formulating a joint prediction task that combines course set selection and grade regression in a single multi-task Transformer model. This architectural framing enables TRACE to capture the mutual dependency between which courses a student takes and how well they perform in them, which has not been done in prior work.

In addition, recent work in learning analytics has increasingly framed student performance prediction as a relational learning problem and has adopted graph neural networks (GNNs) [14] to model complex dependencies among students, courses, and learning behaviors. In this literature, students are typically represented as nodes in a graph, with edges encoding similarity, interaction, or shared attributes. Nodes are then aggregated, and node embeddings updated, all according to various architectural approaches. Graph convolutional networks and graph attention architectures, among other approaches [15, 16], are then used to propagate information across these relational structures to predict academic outcomes.

Several studies construct student-to-student graphs based on demographic similarity, historical grades, or learning behaviors and demonstrate improved predictive accuracy over tabular or sequence-based baselines. For example, [17] model student

similarity using a graph neural network to predict academic performance, while [18] leverage peer interaction graphs derived from online learning environments to capture peer influence effects. Others use multiple graphs, such as [19] which models student interactions and student attributes as two separate graphs, and combines them to make predictions. These approaches highlight the effectiveness of relational inductive biases in modeling student performance, and the flexibility of graph neural network approaches.

We make three contributions:

1. we formulate next-semester academic prediction as a *joint* course-set and grade prediction problem in a single model;
2. we introduce a semester-level concurrency encoding that is permutation-invariant within a term while preserving temporal order across terms; and
3. we propose a set-based multi-task objective (KL for course-set prediction + MSE for grade regression) that avoids order artifacts induced by token-level cross entropy.

In our study we utilize a **TR**ansformer for **A**cademic **C**ourse-grade **E**stimation (TRACE) to predict grade performance based on student major, course history and grade history. In particular we implement two novel approaches, which we show significantly improve grade predictions.

First, we show that simultaneously predicting courses and grades, using an embedding layer to represent courses, leads to significantly higher performance than a simpler model. Indeed, we experiment with a model which still uses course embeddings, but only predicts grades, and note a mean absolute error nearly double that of the model which jointly predicts both, and a mean squared error 3.5 times higher. We also show that predicting just grades, but not courses, leads to significantly reduced performance. Therefore, both factors (simultaneous course and grade predictions, combined with course embeddings) are extremely important.

Despite this, the purpose of joint course prediction is *not* to recommend courses to students, but rather to force the model to learn meaningful course representations. This is analogous to how auxiliary tasks in multi-task learning improve representation quality for the primary task. This is analogous to word embeddings such as those trained by BERT. The primary task was masked word prediction (predicting a missing word based on the surrounding context). However, the training phase had a secondary task of predicting the next sentence, as this leads to stronger sentence relationships, and results in better downstream prediction quality [20].

Second, we maintain the concurrency of courses taken with the semester by utilizing positional encoding, using this to predict all courses in the subsequent semester simultaneously. Doing so allows the model to better understand the relative temporal relationship between courses, but also to predict an entire semester’s worth of courses simultaneously. For contrast, [21], [22], [23] and [24] all use course history to predict future grades, but treat all courses as occurring sequentially, regardless of the semester in which the course was taken. Our approach requires us to introduce a custom loss function which supports this concurrency.

As discussed in the comprehensive review in [25], the majority of the research used course-specific models to predict performance. They note that “it is worth investigating

whether a general prediction model can be developed for use in multiple courses.” Our work is not unique in addressing this problem, but contributes to the literature of general purpose models which can be broadly deployed and easily updated when new courses are implemented.

2 Methods

This study was conducted at an American university, in the context of a LA project which aimed at modeling course performance based on historical course selection, grades and student major. The university collects data on student registration and course grades, and this data was combined with data on student majors. This study obtained anonymized study data consisting of over 5000 students across ten years. All courses were offered in either the fall or spring semester. We sought and received IRB approval for use of this data from the university’s IRB. The university’s informational technology division provided the data.

2.1 Dataset

The dataset consisted of 5326 students enrolled in 360 different courses across 48 majors and 18 semesters. Data extended from the Fall 2014 semester through the Fall 2023 semester, and included all fall and spring semesters in this date range. Students in the data were split into training and testing sets, consisting of 90% and 10% of all students, respectively.

We take the full academic history of the student and create multiple (input, output) pairs from their academic history. We make the first input be the student’s major, courses and grades from semester 1, and the output be the courses and grades from semester 2. Then, assuming the student has at least three semesters of academic history, we make the next input be the courses and grades from semesters 1 and 2, and the output be the courses and grades from semester 3. We repeat this process for the students entire academic history and for all students.

2.2 Model Architecture

In this section we detail our data preparation, and give an overview of the TRACE model architecture. Transformer models were originally developed in the context of machine translation, and thus much of the work around them is described in a natural language processing setting. In this section we give examples showing how there is a natural relationship between language modeling and course and grade prediction.

2.2.1 Data Preparation

Data preparation consisted of data cleaning and standardization. We filtered out students who had completed less than ten courses at the university, as well as those who had taken more than sixty courses. Doing so removed students with insufficient course history for prediction, or who had a course history much longer than is typically expected, perhaps indicating a situation which may not generalize. We label encoded all course names to sequential integers in a random order, ignoring the effect

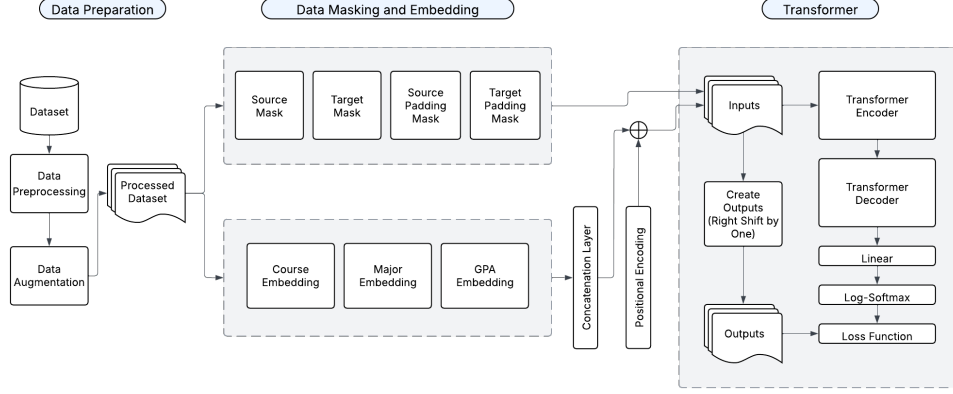


Fig. 1 Overview of model architecture

of course subject or number. This was done purposefully to allow the model to learn latent representations of these courses, without relying on hand-encoded features.

2.2.2 Data Augmentation

Courses that had been taken less than 100 times in total across the entire dataset were all encoded as an `<OTHER>` token. Similarly, majors were label encoded, and majors with less than five hundred students ever having declared them were encoded as an `<OTHER>` token. Grades were converted to standardized GPA values between 0 (F) and 4.3 (A+).

Sequence models are typically standardized [26] in the following way: All sequences are prefixed with a `<SOS>` token to indicate the start of the sequence, appended with a `<EOS>` token to indicate the end of the sequence, and then padded to a common length using `<PAD>` token. Since Transformer models process inputs in fixed-size batches, shorter sequences are typically padded to match the length of the longest sequence in a batch. We do the same for our data, padding all students to have semesters with the same length, where padding is appended to enforce this shared length.

2.2.3 Data Masking

Masking is a fundamental technique employed in Transformer architectures to control information flow and maintain appropriate contextual boundaries. The self-attention mechanism inherently connects all positions to each other, necessitating strategic constraints through various masking approaches to preserve model integrity and enable specific functionalities [27].

Padding masks (also called attention masks) address the variable-length nature of input sequences. To prevent the model from attending to these artificial padding tokens, a padding mask is applied:

$$\text{Mask}_{\text{padding}}(i, j) = \begin{cases} 0 & \text{if position } j \text{ contains a padding token} \\ 1 & \text{otherwise} \end{cases} \quad (1)$$

This binary mask is applied to the attention scores, effectively setting attention scores for padding tokens to $-\infty$, resulting in zero attention weight after softmax [27, 28].

In autoregressive decoding scenarios, such as those employed in the Transformer decoder, each token should only attend to previous tokens in the sequence to prevent information leakage from future positions during training. For example, we would not want our model to know how a student performed in Calculus 2 when predicting their grade in Calculus 1. This is achieved through causal masking (sometimes called target masking or look-ahead masking):

$$\text{Mask}_{\text{causal}}(i, j) = \begin{cases} 0 & \text{if } j > i \\ 1 & \text{if } j \leq i \end{cases} \quad (2)$$

This triangular mask ensures that position i can only attend to positions $j \leq i$, preserving the autoregressive property [27]. In our situation, we utilize attention masks to ensure that the model attends only to previous semesters.

These masking mechanisms collectively enable Transformers to maintain sequence ordering constraints, handle variable-length inputs efficiently, and implement diverse training objectives, contributing significantly to the architecture’s versatility across a wide range of sequence modeling tasks.

2.2.4 Embedding Layers

Transformer models begin by converting discrete input tokens into continuous vector representations through embedding layers. These learned embeddings map tokens from a vocabulary to dense vectors in a high-dimensional space where semantic relationships can be captured [29].

In the original Transformer architecture, separate embedding layers are used for both the encoder and decoder inputs [27]. The embedding dimension is typically set to d_{model} (e.g., 512), which remains consistent throughout the model. Notably, Vaswani et al. multiply these embeddings by $\sqrt{d_{\text{model}}}$ to scale the initial embeddings:

$$\text{Embedding}(x) = \text{Emb}(x) \cdot \sqrt{d_{\text{model}}} \quad (3)$$

This scaling helps maintain appropriate magnitude before adding positional encodings and entering the encoder/decoder stacks [27].

In our work we create separate embedding layers for each major, course, and grade. These embeddings are all independent and learned jointly, allowing the model to learn and leverage the interaction between these factors. While the grade is already numeric,

it is well known that grades are not assigned linearly, and grade distributions often depend on the level of the course [30]. Therefore, passing grades through a small embedding layer allows the model to capture this nonlinearity.

2.2.5 Positional Encoding

In Transformer models, unlike recurrent neural networks, there is no inherent sequential information processing. However, sequence order is crucial for many natural language processing tasks. To address this limitation, positional encoding is introduced to inject information about the position of tokens in the sequence.

The standard approach, as proposed by [27], utilizes sinusoidal functions to create position-dependent representation vectors:

$$\begin{aligned} PE_{(pos, 2i)} &= \sin(pos/10000^{2i/d_{model}}) \\ PE_{(pos, 2i+1)} &= \cos(pos/10000^{2i/d_{model}}) \end{aligned}$$

where pos represents the position in the sequence, i represents the dimension, and d_{model} is the dimensionality of the model. This formulation allows the model to attend to relative positions, as the positional encoding for a fixed offset can be represented as a linear function of the positional encoding at the original position [27].

In our situation, the position is represented by the relative semester in which the student took the course. That is, semesters are encoded as 1 (first semester), 2 (second semester), etc. This allows the model to learn the impact of different temporal lags. Because students typically take multiple courses in a single semester, multiple courses will share the same positional encoding. We use relative semesters rather than absolute semesters (e.g. 1 = “Fall 2014”) in order to allow the model to learn relative temporal importance which is transferrable to future semesters.

2.2.6 Encoder

The Transformer encoder consists of a stack of identical layers, each performing two fundamental operations. First, it analyzes relationships between sequence elements through multi-head self-attention, then processes this information via a position-wise feed-forward network. Both operations employ residual connections followed by layer normalization to facilitate training [31].

The multi-head attention mechanism enables the model to simultaneously examine representations from different viewpoints—analogueous to multiple analysts focusing on different aspects of the same text:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

Each attention head operates independently, calculating relevance scores between sequence elements:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

The attention function uses scaled dot-product attention, computing similarity scores between queries and keys to create weighted combinations of values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The scaling factor $\sqrt{d_k}$ prevents gradient saturation by keeping attention scores within an optimal range for the softmax function [27].

After capturing contextual relationships, the feed-forward network processes each position’s representation individually:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

This dual-stage processing—relationship analysis through attention followed by individual transformation through feed-forward networks—enables the model to capture dependencies regardless of sequential distance, a significant advantage over traditional recurrent architectures [27].

In our context, the encoder attempts to capture the complex course-grade-major relationships and model them for later use in the model. That is, by representing the course and grade history as a latent vector in high-dimensional space, it can model these relationships in a way that is highly flexible, while also not needing to rely on human intervention (e.g. marking one course as a prerequisite for another, marking a course as being a senior-level course, etc).

2.2.7 Decoder

The decoder follows a similar structure to the encoder but includes an additional sub-layer that performs multi-head attention over the output of the encoder. Each decoder layer contains three sub-layers: a masked multi-head self-attention mechanism, a multi-head cross-attention mechanism that attends to the encoder output, and a position-wise feed-forward network.

For the purposes of course and grade prediction, the decoder layer reads in the latent vector created by the encoder layer, and utilizes a multi-head attention mechanism to attend to past courses and grades. The model learns the relevance of course history on grade and course prediction through the training process.

2.2.8 Prediction Layer

After passing through the Transformer decoder, a linear layer is then used to convert the outputs to the proper size. In our case that corresponds to size $2 \cdot N_{\text{course}}$, where N_{course} is the total number of possible courses (including the <OTHER> token). We double this because we predict both the course and the corresponding grade for the course. Note that we *do not* predict the major, as this is already included in the input sequence, thus the output sequence is one term shorter than the input.

2.2.9 Log-Softmax Layer

We utilize the Kullback-Liebler loss function to measure course prediction, as discussed below. This loss function expects the predictions to be in log-space to avoid underflow issues. Therefore we pass logits through the log-softmax layer which computes

$$\text{LogSoftmax}(c_i) = \log\left(\frac{\exp(c_i)}{\sum_j \exp(c_j)}\right)$$

for all courses c_i .

2.3 Training

We group our data into batches of size 32, and train on a single NVIDIA RTX 3090 Ti for 20 epochs. We utilize the AdamW optimizer [32], along with a cosine annealing warm restart learning rate scheduler [33] with $T_0 = T_{\text{mult}} = 5$.

2.3.1 Loss Function

Typically, sequential models are evaluated one token at a time, using either mean squared error (MSE) loss for regression tasks or cross-entropy loss for classification tasks. However, our task is unique in that it involves both regression (grades) and classification (courses). In addition, our classification task, while modeled as a sequence of courses, in fact has multiple tokens occurring simultaneously. This is because students typically take multiple courses in a semester, and there is no natural ordering for these courses. Therefore, a token-by-token evaluation using something like cross-entropy could potentially misclassify predictions as being incorrect, simply because the order predicted did not match the order listed. For example, if the student’s target courses were listed as (c_1, c_2, c_3) , but the model predicted (c_2, c_3, c_1) , then evaluating this token-by-token would show zero correct predictions, which is undesirable. We solve this problem by viewing the courses taken in a given semester as a probability distribution over all possible courses.

We introduce a novel loss function combining KL-divergence over the predicted distribution of courses and MSE for grade regression, designed specifically for set-based prediction. Traditional cross-entropy-based sequence losses are inadequate for unordered output sets such as semester course baskets. Our loss formulation is inspired by general multiset prediction frameworks in ML but has not been previously applied in education data mining [34]. Alternative set-based losses such as Hungarian matching [35] require solving an assignment problem at each training step, adding computational overhead. Chamfer loss [36], while permutation-invariant, treats predictions as point sets without probabilistic interpretation. Our KL-based formulation naturally handles the variable-size course sets while providing interpretable probability distributions over the course catalog.

We treat all courses within a semester as occurring simultaneously utilizing positional encoding, and thus the target is a binary vector corresponding to a one-hot encoding of the courses. We then normalize this so that the entries sum to one by dividing by the total number of courses taken in the semester. This way we can view the target as a probability distribution P over all possible courses. The prediction is also a

probability distribution Q over all possible courses. We then use the Kullback–Leibler divergence (KL divergence) to evaluate the loss:

$$D_{KL}(P \parallel Q) = \sum_{i \in I} P(c_i) \log \left(\frac{P(c_i)}{Q(c_i)} \right).$$

Here I is the indices for all possible courses, and c_i indicates the course in the i 'th position.

We note that course selection can be viewed more simply as a multi-label classification problem, where, given the course history up to that point, the task is simply to predict the next course and grade. Viewed this way, a natural loss function for the course prediction would be binary cross entropy, according to whether or not the correct course is predicted. However, using the KL divergence approach discussed here have several benefits. First, it naturally encodes course load in a given semester. Doing so aligns with the literature on academic momentum discussed in the introduction. In addition, it allows a natural modeling of the interdependency of concurrently taken courses. For example, students taking multiple difficult courses in the same semester may struggle, and viewing course load as a distribution over courses naturally addresses this. Finally, we ran an ablation study comparing the above loss function to binary cross entry for course prediction, and noted grade prediction within 1-2% of each other between the two models for both MAE and MSE. Therefore, while we believe that the KL divergence-based approach has learning analytics-aligned benefits with no prediction drawback, these results suggest the option of utilizing a BCE loss as a reasonable alternative.

For GPA we calculate the MSE loss for the predicted grade against the actual grade. We mask the padding values so that the loss is not calculated for padding courses or for start/end of sequence tokens. We also mask grade predictions for courses not taken, since we are only interested in measuring prediction quality when we have a ground truth to compare the prediction against.

Finally, we balance the relative importance of each of these two loss values (course and grade) through the use of a weighting factor. That is

$$\text{Loss}_{\text{total}} = \text{Loss}_{\text{course}} + \omega \cdot \text{Loss}_{\text{grade}} \quad (4)$$

where $\omega \in \mathbb{R}$ is fixed. Note that D_{KL} is positive but unbounded, and MAE is unbounded and depends on the scale of the grades. Therefore, the precise value of ω will depend heavily on the data, and should be optimized using cross validation during training. Through our experiments we found that a value of ω near one was optimal.

2.3.2 One-Step-Ahead Training

While courses within a semester are taken simultaneously, Transformer models still make predictions sequentially. That is, at any given step the set of courses, grades and semesters, ignoring padding, may look like

$$c = (\langle \text{SOS} \rangle, c_1, c_2, \dots, c_n, \langle \text{EOS} \rangle)$$

$$\begin{aligned}
g &= (0., g_1, g_2, \dots, g_n, 0.) \\
t &= (t_{\langle \text{SOS} \rangle}, t_1, t_2, \dots, t_n, t_{\langle \text{EOS} \rangle})
\end{aligned} \tag{5}$$

where c_i indicates a course, g_i indicates the grade in course c_i , and t_i indicates the semester in which course c_i was taken. However, during inference time, we only know information up to a certain point, and wish to predict future courses and grades. To format this into a sequence learning problem, we utilize one-step-ahead prediction, which truncates each sequence up to position $i \geq 1$, and using the first i entries as the input and the remaining $n - i$ entries as the target. For example, with $i = 2$ we have

$$\begin{aligned}
c_{\text{in}} &= (\langle \text{SOS} \rangle, c_1) & c_{\text{out}} &= (c_2, \dots, c_n, \langle \text{EOS} \rangle) \\
g_{\text{in}} &= (0., g_1) & g_{\text{out}} &= (g_2, \dots, g_n, 0.) \\
t_{\text{in}} &= (t_{\langle \text{SOS} \rangle}, t_1) & t_{\text{out}} &= (t_2, \dots, t_n, t_{\langle \text{EOS} \rangle})
\end{aligned} \tag{6}$$

The model then iterates through all values of i from 1 to n .

3 Results and Discussion

3.1 Comparison Methods

In order to understand both the performance of our model but also the relative importance of various features and architecture choices, we compare our model to the following alternatives and variants:

- **Transformer without course predictions (“OnlyGradesTransformer”):** During training we predict the grades in the courses actually taken, but not the courses taken.
- **Encoder-Decoder LSTM (“EncDecLSTM”):** We utilize an encoder-decoder LSTM with attention. This closely mirrors our Transformer model setup, but replaces the Transformers in the encoder and decoder with unidirectional LSTMs with attention.
- **Unidirectional LSTM (“UniLSTM”):** We also implement a unidirectional LSTM without the encoder-decoder architecture described above. This LSTM also does not use any attention mechanism, and represents a traditional LSTM architecture.
- **Graph Neural Network (“GNN”):** The model operates on a bipartite graph where students and courses constitute two distinct node types, connected by enrollment edges. For each prediction task (predicting a student’s courses and grades for semester t), we construct a subgraph containing the target student’s enrollment history from semesters 1 through $t - 1$, along with enrollments from collaborating students who share at least one course with the target student. This collaborative neighborhood provides the GNN with signals about how similar students performed in related courses.
- **XGBoost (“XGBoost”):** We treat our data as tabular with rows corresponding to a particular (student, semester, course) tuple, where we encode all relevant

data (student ID, semester, course taken, grade in course) as columns with no explicit sequential connections. Note that the inclusion of semester means the model can still model the concurrency of courses taken in the same semester. Therefore, this approach allows us to closely compare the performance of a powerful machine learning algorithm against a model deep learning approach such as an LSTM or Transformer.

For each model we evaluate grade prediction using MSE and mean-absolute error (MAE), and compare the results to the model proposed in this paper (“TRACE”). We conducted cross-validation hyperparameter tuning for the XGBoost model to determine optimal hyperparameters. The results are shown in Table 1.

Because of the relative novelty of graph neural networks, in particular in the use of learning analytics, we provide a brief overview of our architecture and design choices.

3.1.1 Graph Neural Network Architecture

Our GNN operates on a bipartite graph where students and courses constitute two distinct node types, connected by enrollment edges. Predictions, both of courses and grades for semester t , are made by constructing a subgraph containing the target student’s enrollment history from semesters 1 through $t - 1$, along with enrollments from collaborating students who share at least one course with the target student.

Each enrollment edge carries two features: the normalized semester rank (indicating when the course was taken relative to the student’s academic career) and the normalized grade received. These edge features serve an analogous role to the positional encodings in TRACE, encoding temporal information about when each course-grade observation occurred.

Student node features are computed as the concatenation of two components: (1) a learned embedding of the student’s major, and (2) the mean of learned embeddings for all courses in the student’s history. Critically, we do not use learnable per-student parameters, ensuring the model can generalize to unseen students and maintaining parity with TRACE, which similarly lacks student-specific parameters. This aggregation-based initialization allows the model to represent students based on their academic trajectory rather than memorized identities.

Course node features are initialized from a learned embedding table, identical to the course embeddings used in TRACE. Both student and course features are projected to a common dimensionality of 128 to match TRACE’s hidden dimension.

The core of the GNN consists of two layers of edge-conditioned bipartite message passing. Each layer performs bidirectional communication: first from students to courses, then from courses to students. The message passing operation computes messages as $m_{ij} = \text{MLP}([h_i|e_{ij}])$, where h_i is the source node’s features and e_{ij} is the encoded edge features. Messages are aggregated at target nodes via mean pooling, then combined with the target node’s current features through an update MLP. Residual connections and layer normalization stabilize training. We experimented with three, four or five layers for longer distance message passing, but found no improvement in test set performance, at significantly higher computational cost, and with evidence of overfitting.

After two rounds of message passing, each student node has aggregated information from their enrolled courses, and transitively from other students who took those same courses. This two-hop receptive field allows the model to capture collaborative filtering signals—students who took similar courses will develop similar representations.

For the final prediction, we extract the target student’s updated embedding and score it against all possible courses. For each course, we concatenate the student embedding with the course embedding and pass it through two separate prediction heads: a course prediction head outputting a binary logit (trained with binary cross-entropy loss and negative sampling), and a grade prediction head outputting a value in $[0, 1]$ via sigmoid activation (trained with MSE loss). This dual-head architecture mirrors TRACE’s joint prediction of courses and grades.

We note that the GNN model required roughly 50% more training time than TRACE on identical hardware, likely due to the computational overhead of constructing per-prediction subgraphs and message passing across the student-course bipartite structure. This suggests that TRACE offers favorable accuracy-to-compute tradeoffs for institutions with limited computational resources.

3.2 Results

For each of the model and data setups described above, we train the model on the data over twenty epochs. For all models we utilize MSE loss for grade predictions. For models which also predict courses we utilize K-L divergence loss for course predictions. After training we then test on a held-out test set consisting of 10% of all students. We evaluate grade predictions using mean absolute error, in order to understand the error on a $[0, 1]$ -scaled GPA scale, as well as MSE. Grade metrics are computed over the student’s actual enrolled courses. That is, we extract the model’s predicted grade for each course the student actually took, so course prediction accuracy does not influence the reported grade error. This is the natural approach, as we do not have grade data for courses students didn’t actually take.

We see from Table 1 that the Transformer model described in this paper drastically outperforms the XGBoost model. The improvement over the XGBoost model is largely because tree-based models are not naturally suited to sequential learning tasks.

Compared to the two LSTM models, the Transformer model shows a significant improvement, with a MSE approximately 30% lower than the LSTM models, and a MAE approximately 15% lower. We note that the encoder-decoder LSTM uses an attention mechanism, whereas the unidirectional LSTM does not. Regardless, both models have similar performance. This indicates that the key bottleneck is not the attention mechanism, but rather the underlying model architecture. By design, LSTMs process data sequentially, updating their hidden state after each token processed. The Transformer, on the other hand, allows every token to attend to every other (non-masked token). This is likely very important in the context of grade prediction, because relevant courses may have been taken several semester prior, and multiple different courses may all contribute to the prediction in the current course.

Compared to the graph neural network, TRACE has an MSE/MAE 15%-20% lower than the graph neural network. The GNN assumes locality in the graph, namely that a student’s performance is most influenced by their immediate course neighborhood and

by similar students who share that neighborhood. The Transformer model makes no such locality assumption, instead learning which historical observations are relevant through attention. For academic data where prerequisite chains and course sequences matter, the Transformer’s ability to attend to distant but relevant courses may prove advantageous.

Finally, comparing the two Transformer-based models we note a stark difference in performance. The TRACE model achieved a mean absolute error of 0.1339, a 46.4% reduction compared to the ‘OnlyGradesTransformer’’s MAE of 0.2496, and a mean squared error 3.5 times lower. This large difference demonstrates that the Transformer model architecture on its own is not sufficient to guarantee strong performance. A key benefit of the Transformer architecture is the robust latent representation it learns, and if these are not leveraged in the case of courses, then predictions suffer greatly.

Model	MSE	MAE
TRACE	0.0392	0.1339
OnlyGradesTransformer	0.1400	0.2496
EncDecLSTM	0.0530	0.1608
UniLSTM	0.0544	0.1619
GNN	0.0495	0.1640
XGBoost	1.3317	0.8818

Table 1 Overview of results comparing TRACE with baseline models.

As discussed above, the primary purpose of the model is grade prediction, and course prediction is utilized to encourage the model to learn meaningful course vector representations. To assess the quality of course predictions, we computed standard classification metrics for the TRACE model on the test set ($P = 0.5076$, $R = 0.4090$, $F1 = 0.4432$).

These metrics reflect several challenges inherent to course prediction. First, with 360 possible courses in the catalog, the task is substantially more difficult than typical binary or small-cardinality classification problems. Second, many enrollment decisions depend on factors outside the model’s scope, including student preferences, scheduling constraints, advisor recommendations, and choice among equivalent courses (e.g., different courses satisfying the same general education requirement).

As a baseline, random selection of 5 courses from the 360-course catalog would yield an expected precision of approximately 0.014 (5/360), highlighting that even modest metrics represent substantial predictive signal.

Critically, these course prediction metrics demonstrate that the auxiliary task successfully regularizes the learned representations: the model achieves a 46% reduction in grade prediction MAE compared to the OnlyGradesTransformer baseline (Table 1), confirming that imperfect but informative course predictions substantially improve the primary task.

3.2.1 Feature Ablation Analysis

To assess the contribution of individual features, we trained variants of TRACE with different information removed. As discussed above, the ‘OnlyGradeTransformer’ model keeps courses as input, but removes them from prediction. As seen in Table 1, this resulted in a significant drop in performance.

To assess the contribution of student major, we trained a variant of TRACE with major removed from the input. This ablation resulted in marginally improved performance (MSE: 0.0360 vs. 0.0392; MAE: 0.1327 vs. 0.1339), suggesting that major provides no additional predictive signal beyond course history and may introduce slight redundancy.

We interpret this as evidence that a student’s major is largely recoverable from their early course selections. A computer science student, for instance, often enroll in introductory programming and discrete mathematics within their first semesters. These patterns are likely sufficiently distinctive that the model learns major-like groupings implicitly. When major is provided explicitly, the model receives a redundant signal that marginally increases overfitting risk without contributing new information. However, the benefit of keeping an embedding of the major as the first entry in the sequence is that it shows how other student attribute data could be incorporated in a similar manner.

4 Conclusion

A primary goal of this work is to provide a reproducible methodological framework that other institutions can adapt with their own data. The architecture requires no institution-specific feature engineering—course and major embeddings are learned from enrollment data alone, and the model can be retrained on any institution’s historical records without structural modification.

We introduced a novel Transformer-based architecture that jointly predicts both the set of courses a student will take and the grades they will achieve in them. This approach, coupled with a custom loss function, allows the model to learn the complex, non-sequential inter-dependencies between courses, major, and historical performance. We demonstrated that predicting courses taken, as well as grades, is fundamental in downstream tasks. Without prediction of courses, the model fails to learn robust course vector representations, which harms the primary task of grade prediction. Therefore, this work advances the body of learning analytics knowledge by demonstrating that for complex sequential prediction, modeling the structural properties of the sequence (i.e., concurrent semesters) and the inter-relationships of the outputs (i.e., joint course-grade prediction) can be as crucial as the choice of the underlying model architecture itself.

While in our work the only non-course/grade data we had access to was student majors, our approach of prepending embeddings of student majors to the sequence gives a guide for how an institution could implement other student attribute data, such as demographics, high school data, etc. We release our training code to facilitate replication and extension. While single-institution validation is a limitation, our results

establish a performance baseline and architectural template against which future cross-institutional studies can be compared.

4.1 Principal Findings

Our results demonstrate the superiority of this joint prediction approach, as well as the Transformer architecture. The model we developed significantly outperformed comparison models across key metrics. Specifically, our proposed model TRACE, which jointly predicts courses and grades, achieved a mean absolute error nearly half that of an identical Transformer architecture that only predicted grades, and a mean squared error 3.5 times lower. This confirms our hypothesis that forcing the model to learn meaningful course embeddings through the course prediction task provides a richer representation that directly benefits grade prediction accuracy.

Furthermore, when compared to a traditional sequence-to-sequence model using an Encoder-Decoder LSTM with attention, our Transformer model showed marked improvements. Regardless of whether an attention mechanism is utilized or not, and regardless of whether the LSTM is unidirectional or bidirectional, the Transformer model still solidly outperformed it.

Despite their strengths, existing GNN-based approaches typically focus on predicting student outcomes (e.g., grades or pass/fail status) for a fixed course or time window, rather than modeling academic progression across semesters. Courses are usually treated as features or as sources of interaction edges, rather than as first-class prediction targets. Moreover, concurrency of course enrollment is handled implicitly through graph connectivity rather than explicitly as a structural unit of the prediction task. Temporal information, when included, is often incorporated through static snapshots or limited temporal aggregation rather than explicit sequence modeling.

4.2 Early Alert Systems

These findings pave the way for a new class of advising tools that move beyond simple risk-flagging to provide nuanced, data-driven insights into how specific course combinations may impact student success. For example, an early warning system could be constantly running in the background as student grade data is updated each semester. Results could be integrated into student advising. Our model achieves a grade prediction MAE of 0.0392 on a $[0, 1]$ GPA scale, which corresponds to 0.1568 on a typical $[0, 4]$ GPA scale. This means predictions are accurate, on average, to about half of the difference between a typical +/- grade (e.g. B vs B+). accurate enough to serve a meaningful purpose in an early alert system.

While models like this one don't replace human insight, they can help alert faculty and student support staff to key problems before they arise. It has been shown that even minor "nudges" can help students, and that early intervention is key [37] [38] [39] [40]. In spite of this, it is still common for universities to rely on manually entered instructor early alerts [41] [42]. Therefore, systems such as this one can help streamline this process.

4.3 Limitations

Despite these promising results, this study has several limitations. First, the model was trained and tested on data from a single, medium-sized private university. The student demographics, course offerings, and academic rigor at this institution may not be representative of other academic contexts, such as large public universities, community colleges, or institutions in other countries. The model’s performance may not generalize without retraining on institution-specific data.

Second, our model’s features were limited to student major, course history, and grade history. As noted in our introduction, a host of other factors, including socioeconomic status, student engagement, and non-cognitive skills, are known to influence academic success. The exclusion of these variables means our model captures only one dimension of the student experience. Historical grading data may reflect systemic biases related to instructor, demographic, or socioeconomic factors. Our model learns from these historical patterns and may therefore encode and propagate such biases in its predictions. Future work should incorporate fairness auditing to assess whether predictions exhibit disparate accuracy across student subgroups, and explore bias-mitigation strategies during training.

Finally, the model faces a “cold start” problem for new students with no academic history at the university. Similarly, newly introduced courses that are not in the training vocabulary are mapped to an <OTHER> token, which limits the model’s predictive power for novel curricula until sufficient data is collected.

4.4 Future Work

The findings and limitations of this study suggest several avenues for future research. An immediate next step would be to validate the model’s architecture and performance on datasets from a more diverse range of academic institutions to assess its generalizability.

Future work may also explore integrating permutation-invariant architectures such as Set Transformers [43] or DeepSets [44], which may better align with the unordered structure of semester-level course sets.

Future iterations could also aim to incorporate a richer feature set. Integrating data from Learning Management Systems (LMS), such as login frequency, forum participation, and assignment submission times, could provide a more granular view of student engagement. Demographic and financial aid data could also be included to model the impact of non-academic factors, provided that ethical and privacy considerations are carefully managed.

In terms of architecture choices, we compared a Transformer architecture to a graph neural network architecture. Both performed extremely well, especially as compared to common baselines. However, recent work has emerged in Graph Transformer models, which can “automatically and implicitly learn and extract “meta paths” that are important for different downstream tasks”, which show promise for tasks such as those described in this paper [45].

Lastly, exploring the practical application of the model’s interpretability is a key future direction.

5 Data Availability

The datasets generated and/or analyzed during the current study are not publicly available due to the data originating from private data from a private university. However, an anonymized subset of the data is available at <https://github.com/paulsavala/TRACE-grade-prediction>.

6 Declarations

Funding

No funding was received for this work.

Ethics approval and consent to participate

Not applicable.

Consent for publication

Not applicable.

Materials availability

The datasets generated and/or analyzed during the current study are not publicly available due to the data originating from private data from a private university. However, an anonymized subset of the data may be available in an anonymized form from the corresponding author on reasonable request.

Code availability

Code is available at <https://github.com/paulsavala/TRACE-grade-prediction>.

Acknowledgments

We would like to acknowledge the help of St. Edward's University in sourcing this data, and Dean Jon Hodge for supporting this work.

References

- [1] Shafiq, D.A., Marjani, M., Habeeb, R.A.A., Asirvatham, D.: Student retention using educational data mining and predictive analytics: a systematic literature review. *IEEE Access* **10**, 72480–72503 (2022)
- [2] Borsato, G.N., Nagaoka, J., Folley, E.: College readiness indicator systems framework. *Voices in Urban Education* **38**, 28–35 (2013)
- [3] Balfanz, R., Herzog, L., Mac Iver, D.J.: Preventing student disengagement and keeping students on the graduation path in urban middle-grades schools: Early

- p>identification and effective interventions.
- Educational Psychologist*
- 42**
- (4), 223–235 (2007)
- [4] Welch, W.W., Walberg, H.J., Fraser, B.J.: Predicting elementary science learning using national assessment data. *Journal of Research in Science Teaching* **23**(8), 699–706 (1986)
 - [5] Dills, A., Hernández-Julián, R., Rotthoff, K.W.: Knowledge decay between semesters. *Economics of Education Review* **50**, 63–74 (2016)
 - [6] Belanger, K.P., Dills, A.K., Hernández-Julián, R., Rotthoff, K.W.: Class size, course spacing, and academic outcomes. *Eastern Economic Journal* **45**, 301–320 (2019)
 - [7] Quitadamo, I.J., Kurtz, M.J.: Learning to improve: using writing to increase critical thinking performance in general education biology. *CBE—Life Sciences Education* **6**(2), 140–154 (2007)
 - [8] Babad, E., Tayeb, A.: Experimental analysis of students’ course selection. *British Journal of Educational Psychology* **73**(3), 373–393 (2003)
 - [9] Attewell, P., Heil, S., Reisel, L.: What is academic momentum? and does it matter? *Educational Evaluation and Policy Analysis* **34**(1), 27–44 (2012)
 - [10] Ren, Z., Ning, X., Lan, A.S., Rangwala, H.: Grade prediction based on cumulative knowledge and co-taken courses. *International Educational Data Mining Society* (2019)
 - [11] Morsy, S., Karypis, G.: Context-aware non-linear and neural attentive knowledge-based models for grade prediction. *arXiv preprint arXiv:2003.05063* (2020)
 - [12] Khan, M.A.Z., Polyzou, A.: Session-based course recommendation frameworks using deep learning. *International Educational Data Mining Society* (2023)
 - [13] Shao, E., Guo, S., Pardos, Z.A.: Degree planning with plan-bert: Multi-semester recommendation using future courses of interest. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 14920–14929 (2021)
 - [14] Scarselli, F., Gori, M., Tsoi, A.C., Hagenbuchner, M., Monfardini, G.: The graph neural network model. *IEEE transactions on neural networks* **20**(1), 61–80 (2008)
 - [15] Zhang, Z., Cui, P., Zhu, W.: Deep learning on graphs: A survey. *IEEE Transactions on Knowledge and Data Engineering* **34**(1), 249–270 (2020)
 - [16] Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., Sun, M.: Graph neural networks: A review of methods and applications. *AI open* **1**, 57–81 (2020)

- [17] Yu, Y., Fan, J., Xian, Y., Wang, Z.: Graph neural network for senior high student's grade prediction. *Applied Sciences* **12**(8), 3881 (2022)
- [18] Li, H., Wei, H., Wang, Y., Song, Y., Qu, H.: Peer-inspired student performance prediction in interactive online question pools with graph neural network. In: *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pp. 2589–2596 (2020)
- [19] Huang, Q., Zeng, Y.: Improving academic performance predictions with dual graph neural networks. *Complex & Intelligent Systems* **10**(3), 3557–3575 (2024)
- [20] Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171–4186 (2019)
- [21] Polyzou, A., Karypis, G.: Grade prediction with models specific to students and courses. *International Journal of Data Science and Analytics* **2**, 159–171 (2016)
- [22] Okubo, F., Yamashita, T., Shimada, A., Konomi, S.: Students' performance prediction using data of multiple courses by recurrent neural network. In: *International Conference on Computers in Education* (2017)
- [23] Sweeney, M., Lester, J., Rangwala, H.: Next-term student grade prediction. In: *2015 IEEE International Conference on Big Data (Big Data)*, pp. 970–975 (2015). IEEE
- [24] Nachouki, M., Mohamed, E.A., Mehdi, R., Abou Naa, M.: Student course grade prediction using the random forest algorithm: Analysis of predictors' importance. *Trends in Neuroscience and Education* **33**, 100214 (2023)
- [25] Cui, Y., Chen, F., Shiri, A., Fan, Y.: Predictive analytic models of student success in higher education: A review of methodology. *Information and Learning Sciences* **120**(3/4), 208–227 (2019)
- [26] Wu, Y., Schuster, M., Chen, Z., Le, Q.V., Norouzi, M., Macherey, W., Krikun, M., Cao, Y., Gao, Q., Macherey, K., et al.: Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144* (2016)
- [27] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: *Advances in Neural Information Processing Systems*, vol. 30 (2017)
- [28] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V.: Roberta: A robustly optimized bert pretraining

- approach. arXiv preprint arXiv:1907.11692 (2019)
- [29] Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S., Dean, J.: Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems* **26** (2013)
 - [30] Sonner, B.S.: A is for “adjunct”: Examining grade inflation in higher education. *Journal of Education for Business* **76**(1), 5–8 (2000)
 - [31] Ba, J.L., Kiros, J.R., Hinton, G.E.: Layer normalization. arXiv preprint arXiv:1607.06450 (2016)
 - [32] Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. arXiv preprint arXiv:1711.05101 (2017)
 - [33] Loshchilov, I., Hutter, F.: Sgdr: Stochastic gradient descent with warm restarts. arXiv preprint arXiv:1608.03983 (2016)
 - [34] Welleck, S., Yao, Z., Gai, Y., Mao, J., Zhang, Z., Cho, K.: Loss functions for multiset prediction. *Advances in Neural Information Processing Systems* **31** (2018)
 - [35] Kuhn, H.W.: The hungarian method for the assignment problem. *Naval research logistics quarterly* **2**(1-2), 83–97 (1955)
 - [36] Wu, T., Pan, L., Zhang, J., Wang, T., Liu, Z., Lin, D.: Density-aware chamfer distance as a comprehensive metric for point cloud completion. arXiv preprint arXiv:2111.12702 (2021)
 - [37] Smith, B.O., White, D.R., Kuzyk, P.C., Tierney, J.E.: Improved grade outcomes with an e-mailed “grade nudge”. *The Journal of Economic Education* **49**(1), 1–7 (2018)
 - [38] Chen, Q., Okediji, T.O.: Incentive matters!—the benefit of reminding students about their academic standing in introductory economics courses. *The Journal of Economic Education* **45**(1), 11–24 (2014)
 - [39] Gordanier, J., Hauk, W., Sankaran, C.: Early intervention in college classes and improved student outcomes. *Economics of Education Review* **72**, 23–29 (2019)
 - [40] Debrah, M., Timmis, M.A.: Nudging students to success: Investigating the impact of educational nudges on student engagement and outcomes. *Education Sciences* **16**(2) (2026) <https://doi.org/10.3390/educsci16020233>
 - [41] Faulconer, J., Geissler, J., Majewski, D., Trifilo, J.: Adoption of an early-alert system to support university student success. *Delta Kappa Gamma Bulletin* **80**(2) (2013)

- [42] Tampke, D.R.: Developing, implementing, and assessing an early alert system. *Journal of College Student Retention: Research, Theory & Practice* **14**(4), 523–532 (2013)
- [43] Lee, J., Lee, Y., Kim, J., Kosiorek, A., Choi, S., Teh, Y.W.: Set transformer: A framework for attention-based permutation-invariant neural networks. In: *International Conference on Machine Learning*, pp. 3744–3753 (2019). PMLR
- [44] Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R.R., Smola, A.J.: Deep sets. *Advances in neural information processing systems* **30** (2017)
- [45] Hu, Z., Dong, Y., Wang, K., Sun, Y.: Heterogeneous graph transformer. In: *Proceedings of the Web Conference 2020*, pp. 2704–2710 (2020)